

Ветвление (программирование)

Ветвление в программировании — операция, применяющаяся в случаях, когда выполнение или невыполнение некоторого набора команд должно зависеть от выполнения или невыполнения некоторого условия. Ветвление — одна из трёх (наряду с последовательным исполнением команд и [циклом](#)) базовых конструкций [структурного программирования](#).

Основные формы реализации ветвления в императивных языках программирования — **условный оператор** (оператор `if`) и **оператор многозначного выбора** (переключатель, `case`, `switch`). В ранних [языках низкого уровня](#) ветвление реализуется посредством оператора [условного перехода](#).

Условный оператор

Условный оператор реализует выполнение определённых команд при условии, что некоторое логическое выражение (условие) принимает значение «истина» `true`. В большинстве языков программирования условный оператор начинается с ключевого слова `if` (в переводе с [англ.](#) — «если»). Встречаются следующие формы условного оператора — с одной ветвью и двумя.

При выполнении условного оператора с одной ветвью `if <условие> then <команды> end` вычисляется условие, и если оно истинно, то выполняются команды до ключевого слова `end`, в противном случае выполнение программы продолжается со следующей за условным оператором команды. В языках низкого уровня ([ассемблерах](#)) это — единственная доступная форма условного оператора. В некоторых языках для условного оператора с одной ветвью используется специальное ключевое слово (обычно это `then`, в переводе с [англ.](#) — «то»).

При выполнении условного оператора с двумя ветвями `if <условие> then <команды1> else <команды2> end` при истинности условия выполняются команды после ключевого слова `then`, при ложности — команды после ключевого слова `else`. При необходимости проверить последовательно несколько условий возможно каскадирование условных операторов:

```
if условие1
  then команды1
  else if условие2 then команды2
        else if условие3 then команды3
            ...
```

```
else if условиеN+1 then командыN+1
    else команды end;
```

В этом случае условия будут проверяться последовательно, и как только встретится истинное, будет выполнен соответствующий набор команд и исполнение перейдёт к команде, следующей за условным оператором. Если ни одно из условий не окажется истинным, выполняются команды из ветви `else`.

Ряд языков программирования содержит специальную конструкцию для каскадирования условных операторов, позволяющую записать множественное ветвление несколько компактнее и менее подверженную ошибкам написания:

```
if условие1 then команды1
elseif условие2 then команды2
elseif условие3 then команды3
...
else команды end;
```

порядок выполнения этого оператора в точности соответствует вышеприведённому каскаду простых операторов if-then-else, а отличие чисто формальное: вместо вложенных нескольких условных операторов эта конструкция является единым целым и содержит дополнительное ключевое слово `elsif`, требующее после себя очередное условие.

Реализации

[Паскаль](#) унаследовал от [Алгола-60](#) синтаксис, согласно которому в ветвях условного оператора может быть помещена только одна команда. Поэтому для размещения там большего количества команд они группируются в составной оператор с помощью пары ключевых слов `begin` и `end`. Ветвь `else` необязательна. `begin` и `end` необходимы, только если операторов несколько (например, из соображений [единообразия оформления кода](#)). В примере — оператор выбора в Паскале:

```
If условие
then
    begin
        операторы;
    end
else
    begin
        операторы;
    end;
```

Необходимость условного оператора в Алголе и Паскале с момента появления была объектом критики. Критики говорили, что многочисленные составные операторы загромождают программу, мешают нормальной расстановке отступов и провоцируют ошибки (если в последней ветви оператора `if` забыть составной оператор там, где он необходим, то компилятор ничего не заметит, но при выполнении программы из группы операторов, которые должны выполняться в этой ветви, по условию будет выполняться только первый, все остальные — всегда). Следующие поколения языков — потомков Алгола попытались избавиться от этого недостатка. В их числе три широко известных языка: [Алгол-68](#), [Модула-2](#) и [Ада](#). Конструкция оператора `if` в них практически одинакова, с точностью до отдельных ключевых слов:

- Алгол-68:

```
if условие
...
fi;
```

- Модула-2

```
IF условие1 THEN команды1
ELSE IF условие2 THEN команды2
...
ELSE командыN
END;
```

- Ада

```
if условие1 then команды1
else if условие2 then команды2
...
else командыN
end if;
```

Во всех случаях «командыX» — любое число операторов разделённых точкой с запятой. Во всех случаях все ветви условного оператора, кроме первой (ветви «`then`») необязательны и могут быть пропущены. Если ветвь «`else`» отсутствует и ни одно из условий не выполняется, то управление передаётся на команду, следующую за ключевым словом завершения условной конструкции (`END`, `FI` или `END IF`).

[Си](#) и [C++](#) (а вслед за ними и [Java](#), [C#](#), [PHP](#) и множество других языков) имеют условный оператор, структурно аналогичный Паскалю. Отличие состоит в том, что условие должно

быть записано в круглых скобках, а вместо ключевых слов `begin` и `end` используются фигурные скобки `{}`:

```
if (<условие>)  
{  
<операторы>  
}  
else  
{  
<операторы>  
}
```

В [Nemerle](#), в отличие от большинства языков, где оператор `if` может иметь как одну, так и две ветви, оператор `if` (синтаксис полностью аналогичен языку Си) обязан иметь две ветви. Условный оператор с одной ветвью начинается с ключевого слова `when`, кроме того, в языке имеется ещё один условный оператор — `unless`, представляющий собой «обратный when» — в нём команды условной ветви выполняются, если условие ложно.

```
when (условие){ операторы }  
unless (условие) { операторы }
```

В [Форте](#) условный оператор имеет отличный от вида в других языках вид, из-за постфиксной формы записи и стековой организации. Условный оператор состоит из трёх слов `IF ELSE THEN`^[1].

```
<условие>  
IF  
<выражение_1_если_условие_истинно>  
ELSE  
<выражение_2_если_условие_ложно>  
THEN
```

Здесь `<условие>` просто помещает значение на вершину стека, `IF` анализирует флаг, и если:

- он не равен нулю, то выполняются выражения до `ELSE` или `THEN`;
- если он равен нулю, то выполняется выражения между `ELSE` и `THEN`.

При отсутствии `ELSE` получается селектор с одной ветвью: выражения между `IF` и `THEN` выполняются только при ненулевом значении флага.

Фортран изначально имел только **арифметический IF**, в котором в зависимости от знака выражения производился переход на одну из трёх меток. Например, часть кода подпрограммы решения квадратного уравнения:

```
DN = B*B - 4*A*C
IF(DN) 90,10,10
10 D = SQRT(DN)
X1 = (-B + D) / (2*A)
X2 = (-B - D) / (2*A)
```

Затем были добавлены логические (булевские) выражения и логический IF с одним оператором, вычисляемый GOTO, позже — структурный IF (с несколькими условиями), например:

```
DN = B*B - 4*A*C
IF(DN.GE.0) THEN
    D = SQRT(DN)
    X1 = (-B + D) / (2*A)
    X2 = (-B - D) / (2*A)
ELSE
    ! (код для случая отрицательного дискриминанта не приводится)
END IF
```

Perl поддерживает структурный if с несколькими условиями, а также модификаторы оператора (statement modifiers), которые записываются после выполняемой части оператора. Например, два следующих примера идентичны по функциональности:

```
if ($a == 0) {
    ++$zero_count;
}
```

```
++$zero_count if $a == 0;
```

Вместо if можно писать unless, что приводит к инверсии значения условного выражения перед проверкой. То же самое действие через unless:

```
++$zero_count unless $a != 0;
```

Для составного оператора (блока) допустима только структурная форма, но не модификатор. Например:

```

if ($a == 0) {
    ...
} else if ($a > 0) {
    ...
} else {
    ...
}

```

Завершающее ключевое слово не нужно, за счёт требования обязательного оформления операторов под условиями в блоки {...}.

Не существует аналога слова unless для веток elsif.

Erlang использует два условных оператора — if и case. Оба имеют результирующее значение, которое равно значению последнего оператора в выполненной ветке и может быть использовано (назначено имени, передано в функцию...), поэтому в нём нет отдельного **тернарного условного оператора**. В операторе case выполняется **Сопоставление с образцом**, с возможностью дополнительных условий на значения в сравниваемом, а в операторе if — только проверка условий. В условиях (guard tests) допускается ограниченное множество операций и встроенных функций.

Пример на case (удаление записи о событии из дерева времён):

```

{NewEBW, NewEBN} = case dict:find(EBN, Node) of
    error ->
        {EBW, EBN};
    {ok, Expiry} ->
        {gb_trees:delete_any({Expiry, Node}, EBW),
         dict:erase(Node, EBN)}
end,

```

Примеры на if:

```

if
    NeedLater ->
        erlang:send_after(
            trunc(1000*(1+Elapsed)),
            self(), i_apply_nodes_portion);
    true ->
        nop
end,

```

```

After2 = if
    %% If it was too far ago, schedule timeout

```

```
immediately.
```

```
After1 =< ?EXPIRY_STEP_MIN -> ?EXPIRY_STEP_MIN;  
After1 >= ?EXPIRY_STEP_MAX -> ?EXPIRY_STEP_MAX;  
true -> After1  
end,
```

Оператор множественного выбора

Конструкция переключателя имеет несколько (две или более) ветвей. Переключатель выполняет одну заданную ветвь в зависимости от значения вычисляемого ключевого выражения. Принципиальным отличием этой инструкции от условного оператора является то, что выражение, определяющее выбор исполняемой ветви, возвращает не логическое, а целое значение, либо значение, тип которого может быть приведён к целому. В некоторых языках допускается использовать в переключателе выражения некоторых типов, не приводимых к целому (например, текстовые строки).

Прототипом современной синтаксической конструкции была используемая в старых языках программирования команда перехода по вычисляемой метке. В этой команде указывалось выражение-селектор, возвращающее целое значение, и набор меток. При выполнении команды вычислялось выражение, а его значение использовалось как номер метки (в списке команды), на которую производился переход. Такие конструкции были, например, в языках программирования [Фортран](#) («вычисляемый GOTO») и [Бейсик](#). Привлекательной стороной конструкции является её достаточно высокая эффективность: для определения нужной ветви (метки перехода) не требуется последовательно сравнивать результат выражения-селектора со многими значениями, достаточно записать в память массив команд безусловного перехода с нужными адресами, чтобы при выполнении команды вычислять нужный элемент непосредственно из значения выражения. При этом скорость выполнения команды не зависит от количества меток. В современных языках реализация оператора-переключателя также часто выполняется в виде таблицы перехода, состоящей из команд безусловного перехода на соответствующие фрагменты кода. Вычисляемое выражение преобразовывается в значение сдвига по таблице перехода, определяющее выполняемую команду. В языках, где выражение-селектор может иметь нецелое значение, напрямую вычислить нужную ветвь конструкции переключателя можно далеко не всегда, поэтому в них используются другие методы оптимизации исполнения.

В современных языках программирования высокого уровня команда-переключатель обычно имеет имя `switch` (в переводе с [англ.](#) — «переключатель») либо `case` (в переводе с [англ.](#) — «случай»). Однако, выбор по вычисляемой метке может сохраняться в современных языках программирования низкого уровня, например, инструкция JL

языка программирования STL для [программируемых логических контроллеров S7-300](#) и [S7-400](#), выпускаемых [Siemens](#).

Например, в языке Си синтаксис команды следующий:

```
switch (i)
{
case 0:
case 1: // последовательность операторов
    break;
case 2: // последовательность операторов
    break;
default:
    ;
}
```

Здесь *i* — выражение-селектор, которое обязано иметь приводимый к целому тип, каждая ветвь исполнения начинаются с ключевого слова `case`, за ним следует значение выражения, при котором должна выполняться данная ветвь. Интересной особенностью языка Си является то, что в нём переключатель трактуется именно как команда перехода по вычисляемой метке, а роль меток играют заголовки ветвей (`case значение :`). Чтобы после завершения кода ветви произошёл выход из оператора переключателя, используется специальная команда `break`. Если такой команды в ветви нет, после исполнения кода выбранной ветви начнётся исполнение кода следующей за ней. Эта особенность может использоваться для оптимизации, хотя может служить причиной труднообнаруживаемых ошибок (если программист случайно пропустит `break`, компилятор не выдаст ошибки, но программа будет выполняться неверно). Ветвь `default` исполняется тогда, когда среди прочих ветвей не нашлось ни одной подходящей.

Синтаксис команды-переключателя Си унаследован множеством языков, но семантика его не всегда полностью аналогична Си. Например, в C# допускается использовать выражение-селектор строкового типа и соответствующие метки.

Особенности вычисления логических выражений

На порядок исполнения программы с условными операторами может существенно влиять принятая в языке логика вычисления условных выражений. Когда условие представляет собой сложное логическое выражение, к примеру « $f(x) > 0$ И $g(y) > 0$ », существует две стратегии вычисления его результата:

- Полное вычисление: вычислить $f(x)$, $g(y)$, произвести сравнение результатов с нулём, затем выполнить операцию «И» для результатов. Так поступает, к примеру, Visual Basic.

- Неполное вычисление: вычислить $f(x)$, сравнить значение с нулём. Если результат сравнения — «истина», то вычислять остальную часть выражения. Если же первое условие ложно, то пропустить второе условие, в том числе вычисление входящего в него $g(y)$, так как для операции «И» при ложности одного из операндов всё выражение заведомо ложно.

Второй вариант является наиболее распространённым для промышленных языков (в частности, для Алгола, Фортрана, C++, C, Java, JavaScript, ECMAScript, JScript, C#, Python). В этих языках действует жёсткое правило: «Логическое выражение всегда вычисляется слева направо и его вычисление останавливается сразу же, как только результат всего выражения становится определённым». Это означает, что если выражение состоит из нескольких подусловий, объединённых оператором «И» (AND), то вычисление выражения прекратится, как только одно из подусловий окажется ложным (так как «ложь AND любое значение» в результате всегда даёт «ложь»), и, наоборот, если несколько подусловий объединены оператором «ИЛИ» (OR), вычисление прекратится после первого же истинного подусловия, поскольку в этом случае всё выражение истинно, независимо от дальнейших вычислений. А вот выражение, содержащее оператор «Исключающее ИЛИ» (XOR) неполному вычислению не поддаётся, поскольку в нём одно из значений не может определить результат вычисления всего выражения.

Языки [Ада](#) и [Erlang](#) используют разные ключевые слова для этих вариантов: слова `and` и `or` означают полное вычисление, а `and then`, `or else` (Ада), `andalso`, `orelse` (Erlang) — неполное. В Erlang `andalso` и `orelse` менее приоритетны, чем операции сравнения, что позволяет избежать скобок вокруг элементарных условий. Язык Visual Basic.NET имеет похожие ключевые слова: `AndAlso` и `OrElse`.

Фиксированный порядок вычисления подусловий (логическое выражение всегда вычисляется слева направо) вводится для того, чтобы иметь возможность управлять порядком вычисления выражения и помещать в него сначала те условия, которые должны вычисляться в первую очередь. Этим, кстати, логические выражения отличаются от арифметических, для которых, в большинстве языков, порядок вычисления подвыражений (если только он не определён приоритетом и ассоциативностью операций) языком не задаётся и в разных случаях может быть различным.

Выбор именно такой логики исполнения связан с тем, что она позволяет упростить логические выражения, в которых используются зависимые элементы. Классический пример — линейный поиск в массиве:

```
// Поиск в массиве целых чисел на языке Паскаль  
// Параметры - искомое значение и открытый массив целых чисел  
// Результат - индекс найденного элемента или -1 в случае, если  
элемент не найден
```

```

function Find(e: integer; var a: array of integer): integer;
var
  i: integer;
begin
  i := 0;
  while (i <= High(a)) AND (a[i] <> e) do inc(i); // !!!
  if i <= High(a)
  then Find := i
  else Find := -1;
end;

```

Алгоритм, реализуемый программой, совершенно очевиден, но в реализации есть одна тонкость (см. строку, помеченную восклицательными знаками): условие цикла состоит из двух частей, связанных оператором AND. Первое подусловие проверяет, не вышел ли индекс *i* за пределы массива, второе — не равен ли текущий элемент массива искомому значению. Если массив не содержит искомого значения, то после проверки последнего элемента значение переменной *i* увеличится на единицу; на следующей итерации первое подусловие окажется ложным и цикл завершится без проверки второго подусловия. Если бы логические выражения вычислялись полностью, то при отсутствии искомого элемента в массиве после последней итерации происходила бы ошибка: попытка определить *a[i]* вызывала бы некорректное обращение к памяти.

Следует обратить внимание, что, кроме неполного вычисления значения выражения, здесь также играет существенную роль фиксированный порядок вычисления подусловий: поскольку проверка выхода за границу массива записана первой, она всегда будет выполняться раньше, чем проверка достижения искомого значения. Если бы порядок вычисления подвыражений был неопределённым, гарантировать правильность приведённого фрагмента программы было бы невозможно.

При полном вычислении логических выражений приведённый алгоритм пришлось бы записать примерно в следующем виде:

```

// Поиск в массиве целых чисел на языке Паскаль
// Гипотетический вариант при полном вычислении логических выражений
// Параметры - искомое значение и открытый массив целых чисел
// Результат - индекс найденного элемента или -1 в случае, если
элемент не найден
function Find(e: integer; var a: array of integer): integer;
var
  i: integer;
  f: boolean; // дополнительная переменная - флаг завершения цикла
begin
  i := 0;
  f := false;
  while not f do
    if i > High(a) then f := true

```

```

    else if a[i] = e then f := true
    else inc(i);
if i <= High(a)
    then Find := i
    else Find := -1;
end;

```

Как видим, пришлось искусственно задать порядок вычисления условий выхода и ввести дополнительную переменную. Именно для того, чтобы избежать подобных трюков, и введено неполное вычисление логических выражений.

Примечание: Код изложенный выше, является примером использования оператора IF, но не более. Этот код нельзя использовать как правило для написания алгоритмов на языке Паскаль.

Оптимальный алгоритм для поиска числа в массиве:

```

function Find(e: integer; var a: array of integer): integer;
var
    i: integer;
begin
    Result := -1;
    for i := Low(a) to High(a) do begin
        if a[i] = e then begin
            Result := i;
            Break;
        end;
    end;
end;
end;

```

Примечания

- В Forth существует оператор `<условие> ? DUP <выражение>`, который дублирует выражение, если условие истинно

У этой статьи есть 3 проблемы, помогите их исправить:

- В статье не хватает ссылок на источники (см. рекомендации по поиску).
- Достоверность этой статьи поставлена под сомнение.
- Статья этой статьи неэнциклопедичен или нарушает нормы литературного русс

[\[скрыть\]](#)

[Узнать больше](#)

[Узнать больше](#)

[Узнать больше](#)

[Сообщить об ошибке](#)